

Object Oriented Programming (Classes-I)

12 avril 2024

Table of contents

- 1 Classes
- 2 Objects
- 3 Abstraction
- 4 String Data Members

Introduction

- A **class** is a user-defined data type.
- The variables and functions declared within a class are called **members of the class**.
- The variables are called **data members** and the functions are called **member functions**. The functions are sometimes referred to as **methods**.
- Once a class is defined, we can declare variables of the class type. These variables are also called **objects** of the class.
- A class definition ends with a semicolon.

Defining a Class

- To define a class, the keyword *class* is used. The general form to declare a class is :

```
1  class className{  
2  
3      //  data members  
4      //  member functions  
5  
6  };
```

- When we create a class, all of its members are hidden by default and are not accessible outside the class. They are said to be the private members of the class and private members can only be accessed within the same class.

Defining a Class

- We can also manually declare the data members of a class as private using the following statement :

`private :`

- All the members following this label will be private.
- For the members of a class to be accessible outside the class, we must declare them to be public members of the class by using the following statement :

`public :`

- All class members that follow this statement in the class definition will be public up to the point where another specifier appears.
- The keywords `private` and `public` are access specifiers. An access specifier is always followed by a colon.
- All the class members that follow an access specifier in the class definition will obey the specifier until another access specifier is defined.

Defining a Class

Generally, we declare members of a class as follows :

```
1  class className{  
2      private:  
3          // data members  
4      public:  
5          // member functions  
6  };
```

This is equivalent to

```
1  class className{  
2      // data members  
3      public:  
4          // member functions  
5  };
```

Defining a Class

- A class normally consists of one or more member functions that manipulate the data members of the class.
- Data members are declared inside a class definition but outside the bodies of the class's member function definitions. Data member declared *private* are accessible only to member functions of the class.
- Declaring data members with access specifier *private* is known as data hiding. When a program creates an object, data members are *encapsulated* (hidden) in the object and can be accessed only by member functions of the object's class.
- A class's data members cannot be initialized where they're declared in the class body (With the exception of *static const int* data members).
- Variables declared in a function definition's body are known as local variables. A local variable cannot be accessed outside the function in which it's declared.

The *set* and *get* Functions

- Classes often provide public member functions to allow clients of the class to *set* (i.e., assign values to) or *get* (i.e., obtain the values of) *private* data members. These member function names need not begin with *set* or *get*, but this naming convention is common.
- Set functions are sometimes called *mutators* (because they mutate, or change, values), and get functions are also called *accessors* (because they access values).
- The *set* and *get* functions allow a client to interact with an object, but the object's private data remains safely encapsulated (i.e., hidden) in the object itself.
- The *set* and *get* functions of a class also should be used by other member functions within the class to manipulate the class's private data, although these member functions can access the private data directly.

UML Class Diagram

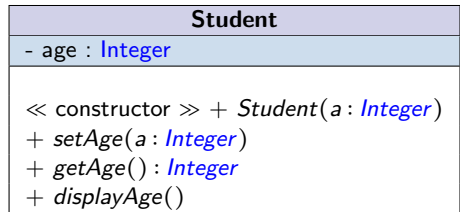
- UML is a standardized graphical language used by software developers to represent their object-oriented systems.
- In the UML, each class is modelled as a rectangle with three compartments.
 - The top compartment contains the class's name centred horizontally and in boldface type.
 - The middle compartment contains the class's attributes, which correspond to data members in C++.
 - The bottom compartment contains the class's operations, which correspond to member functions in C++.
- The UML represents data members as attributes by listing the attribute name, followed by a colon and the attribute type.
- For a private data member, the class diagram lists a minus sign (-) in front of the corresponding attribute's name. The minus sign in the UML is equivalent to the private access specifier in C++.

UML Class Diagram

- The plus sign (+) in front of the operation name indicates a public operation in the UML (i.e., a public member function in C++).
- The UML models a parameter by listing the parameter name, followed by a colon and the parameter type in the parentheses following the operation name. The UML has its own data types similar to those of C++.
- The UML indicates the return type of an operation by placing a colon and the return type after the parentheses following the operation name.
- For operations that do not return values (i.e., they return *void* in C++), the UML class diagram does not specify a return type after the parentheses of these operations.
- The UML is language independent so its terminology does not exactly match that of C++.

UML Class Diagram

```
1 class Student{  
2     int age;  
3  
4     public:  
5         Student(int a);  
6         void setAge(int a);  
7         int getAge();  
8         void displayAge();  
9 };
```



Constructors

- A class constructor is a special kind of function in a class.
- It is automatically called when an object of the class is created.
- It initializes the object.
- A constructor has the same name as the class in which it is defined.
- A constructor does not return any value and therefore has no return type. Even void is not allowed to specify its return type.
- The general form of a constructor is :

```
1 className(parameter-list){  
2     // initialization of data members  
3 }
```

- Here *parameter – list* initializes the data members of a class.

The Default Constructor

- Every class has at least one constructor because an object of class is always created using a constructor.
- If we do not define a constructor for our class then the compiler will supply a default constructor that will be used to create objects of our class. The default constructor has no parameter.
- When we declare a class, the compiler supplies a default constructor automatically. This automatic default constructor exists as long as we do not define a constructor for the class our self.
- The default constructor generated by compiler does nothing. It does not initialize the data members of the object that is created.
- A programmer can define a default constructor of his class as well.
- Note that a default constructor can be called with out specifying an *argument – list*. It does not even need parentheses.

Defining Objects

- To define a class object, the general form is :

```
className objectName(argument-list);
```

- When an object of a class is created, the class constructor is automatically called. The values specified in *argument – list* are passed as arguments to the constructor and are used to initialize the data members of the object.
- The number of values specified in the argument-list must be the same as in constructor's *argument – list*.
- Classes *encapsulate* (i.e., wrap) attributes and member functions into objects. Each object of a class maintains its own copy of data members in memory.

Accessing Class Members

- We can not access class members without the object of that class.
- To access member of a class object, member access operator is used. The general form to access a particular member is :

```
objectName.member;
```

- Here **member** can be a data member as well as a member function. Member functions defined inside the class definition are *inline* functions by default.

Example 1 : Accessing public Members

```
1
2 class Student{
3     public:
4         int age;
5         void setAge(int a) {age = a;}
6 };
7
8 int main(){
9
10     Student s;
11     s.setAge(20);
12     cout<<"Student Age = "<<s.age<<endl;
13
14     return 0;
15 }
```

Student Age = 20

Example 2 : Accessing public Members

```
1
2 class Student{
3     public:
4         int age;
5         int getAge(int a) {return age;}
6 };
7
8 int main(){
9
10     Student s;
11     s.age = 20;
12     cout<<"Student Age = "<<s.getAge()<<endl;
13
14     return 0;
15 }
```

Student Age = 20

Example 3 : Accessing private Members

```
1
2 class Student{
3     int age;
4     void setAge(int a) {age = a;}
5 };
6
7 int main(){
8
9     Student s;
10    s.setAge(20);    // Error!
11    cout<<"Student Age = "<<s.age<<endl; // Error!
12
13    return 0;
14 }
```

Example 4 : *set* and *get* Functions

```
1 class Student{
2     int age;
3     public:
4         void setAge(int a) {age = a;}
5         int getAge() {return age;}
6 };
7
8 int main() {
9
10     Student s;
11     s.setAge(20);
12     cout<<"Student Age = "<<s.getAge()<<endl;
13
14     return 0;
15 }
```

Student Age = 20

Example 5 : Default Constructor

```
1 class Student{
2     int age;
3     public:
4         Student(){cout<<"Constructor .."<<endl;}
5     void setAge(int a) {age = a;}
6     int getAge() {return age;}
7 };
8
9 int main() {
10
11     Student s;
12     cout<<"Student Age = "<<s.getAge()<<endl;
13     s.setAge(20);
14     cout<<"Student Age = "<<s.getAge()<<endl;
15     return 0;
16 }
```

```
Constructor ..
Student Age = 533381769
Student Age = 20
```

Example 6 : Initialization using Constructor

```
1 class Student{
2     int age;
3     public:
4         Student(){age = 10;}
5         void setAge(int a) {age = a;}
6         int getAge() {return age;}
7 };
8
9 int main() {
10
11     Student s;
12     cout<<"Student Age = "<<s.getAge()<<endl;
13     s.setAge(20);
14     cout<<"Student Age = "<<s.getAge()<<endl;
15     return 0;
16 }
```

Student Age = 10
Student Age = 20

Example 7 : Parametrized Constructor

```
1 class Student{
2     int age;
3     public:
4         Student(int a){age = a;}
5     void setAge(int a) {age = a;}
6     int getAge() {return age;}
7 };
8
9 int main() {
10
11     Student s(10);
12     cout<<"Student Age = "<<s.getAge()<<endl;
13     s.setAge(20);
14     cout<<"Student Age = "<<s.getAge()<<endl;
15     return 0;
16 }
```

Student Age = 10
Student Age = 20

Example 8 : Illegal use of Constructor

```
1 class Student{
2     int age;
3     public:
4         Student(int a){age = a;}
5     void setAge(int a) {age = a;}
6     int getAge() {return age;}
7 };
8
9 int main() {
10
11     Student s;    // Error!
12     cout<<"Student Age = "<<s.getAge()<<endl;
13     s.setAge(20);
14     cout<<"Student Age = "<<s.getAge()<<endl;
15     return 0;
16 }
```

Example 9 : Multiple Objects

```
1 class Student{
2     int age;
3     public:
4         Student(int a){age = a;}
5         void setAge(int a) {age = a;}
6         int getAge() {return age;}
7 };
8 int main() {
9     Student s1(10);
10    cout<<"Student Age = "<<s1.getAge()<<endl;
11    cout<<"Student Age = "<<s2.getAge()<<endl;
12    s1.setAge(30);
13    cout<<"Student Age = "<<s1.getAge()<<endl;
14    cout<<"Student Age = "<<s2.getAge()<<endl;
15    return 0;
16 }
```

Student Age = 10
Student Age = 20
Student Age = 30
Student Age = 20

Separating Interface from Implementation

- Interfaces define and standardize the ways in which people and systems interact with one another. For example, an ATM's controls serve as an interface between the ATM's users and its internal components.
- The controls allow users to perform a limited set of operations (such as money withdraw, check balance, and money transfer). Various ATMs may implement these operations differently. Some provide push buttons, some provide touch screen and some support voice commands.
- The *interface* specifies what operations a ATM permits users to perform but does not specify how the operations are implemented inside the ATM.
- Similarly, the *interface* of a class describes what services a class's clients can use and how to request those services, but not how the class carries out the services.
- Separating the interface from the implementation details is called *abstraction*.

Member Functions Outside the Class

- Member functions can be defined inside a class, but it is good practice to define the functions outside the class definition.
- Declaring member functions inside a class definition and then defining these member functions outside the class definition separates the *interface* of a class from its *implementation*.
- When a member function is defined after its corresponding class definition, the function name is preceded by the class name and the binary scope resolution operator (`::`) i.e.,

```
1 returnType  className::funName(parameter-list)
2 {
3     //  body of function
4 }
```

Member Functions Outside the Class

- The member function defined outside the class definition still has a class scope i.e., its name is known only to the other members of the class unless referred to via an object of the class.
- Note that if a member function is defined in a class definition, the member function is automatically inlined. Member functions defined outside a class may be made inline explicitly using the keyword *inline*.
- A constructor definition can also be placed outside the class definition. The constructor is declared in class definition and defined after its corresponding class definition. To define a constructor outside the class, the general form is :

```
1  className::className(parameter-list)
2  {
3      //  body of constructor
4  }
```

Example : Member Functions Outside the Class

```
1 class Student{
2     int age;
3     public:
4         Student(int);
5         void setAge(int);
6         int getAge();
7 };
8 int main() {
9     Student s(10);
10    cout<<"Student Age = "<<s.getAge()<<endl;
11    s.setAge(20);
12    cout<<"Student Age = "<<s.getAge()<<endl;
13    return 0;
14 }
15 Student::Student(int a){age = a;}
16 void Student::setAge(int a) {age = a;}
17 int Student::getAge() {return age;}
```

Student Age = 10
Student Age = 20

Exercise

- Create a class called *Box* that uses three variables of type *double* to calculate the volume of a box. Initialize an object of type *Box* to specific dimensions, then calculate the volume it represents, and print out the result
- Create a class *Rectangle* with attributes *length* and *width*. Provide member functions that calculate the perimeter and the area of the rectangle. Also, provide *set* and *get* functions for the *length* and *width* attributes. The *set* functions should verify that *length* and *width* are each *double* numbers larger than 0.0 and less than 20.0.
- Model the class diagrams for the above two exercises.

String Data Members

Three common ways to store strings in C++ are :

① Character Arrays (C-Style Strings) :

- (a) Character arrays, also known as C-style strings, are arrays of characters terminated by a null character '\0'.
- (b) They are declared as arrays of characters, where the size is typically one more than the length of the string to accommodate the null character. For example,

```
char str[10] = "Hello";
```
- (c) Manipulating C-style strings involves using functions from the `<cstring>` header, such as :

```
strcpy(), strlen(), strcat().
```
- (d) They have a fixed size and can lead to buffer overflow if not handled carefully.

String Data Members

② Character Pointers (C-Style Strings with Pointers) :

- (a) Character pointers are pointers that point to a sequence of characters in memory.
- (b) They are often used to dynamically allocate memory for strings using *new*.
- (c) They offer more flexibility than fixed-size character arrays but require manual memory management and can lead to memory leaks if not handled properly.
- (d) Like character arrays, they are null-terminated and manipulated using functions from `<cstring>`.

String Data Members

3 The *string* Data Type

- (a) The *string* data type is a part of the C++ Standard Library and provides a more convenient and safer way to handle strings compared to C-style strings.
- (b) It is a class template provided by the `<string>` header and offers a wide range of member functions for string manipulation.
- (c) It automatically manages memory allocation and deallocation, avoiding common pitfalls associated with C-style strings like buffer overflow and memory leaks.
- (d) It supports various operations such as concatenation, substring extraction, searching, and more, making string manipulation easier and less error-prone.

Example : A Simple Code

Object Orientation

```
1 class Student{
2     char* name;
3     public:
4         void setName(char* n){name = n;} //2-Problems
5         char* getName(){return name;} // Problem?
6 };
7
8 int main(){
9
10     Student s;
11     char array[20] = "Object Orientation";
12     s.setName(array);
13     cout<<s.getName()<<endl;
14
15     return 0;
16 }
```

Problem in the Setter Function

Object Orientation
Xbject Orientation

```
1 class Student{
2     char* name;
3     public:
4         void setName(char* n){name = n;} // n will destroy
5         char* getName(){return name;}
6 };
7
8 int main(){
9     Student s;
10    char array[20] = "Object Orientation";
11    s.setName(array);
12    cout<<s.getName()<<endl;
13    array[0] = 'X'; // Encapsulation violation
14    cout<<s.getName()<<endl;
15    return 0;
16 }
```

Solution

```
1 class Student{
2     char* name;
3     public:
4         void setName(char* n); // Prototype
5         char* getName(){return name;}
6         ~Student(){delete name;}
7 };
8 int main(){
9     Student s;
10    char array[20] = "Object Orientation";
11    s.setName(array);
12    cout<<s.getName()<<endl;
13    array[0] = 'X'; // Attempt to violate
14    cout<<s.getName()<<endl;
15    return 0;
16 }
```

Object Orientation
Object Orientation

Solution (Cont.)

```
1 void Student::setName(char* n){ // Definition
2     int i, length = 1;
3
4     // Implementation of strlen(n)
5     for(i = 0; n[i] != '\0'; ++i) length++;
6
7     // Implementation of strcpy(name, n)
8     name = new char[length];
9     for(i = 0; i < length; ++i) name[i] = n[i];
10    name[length] = '\0';
11 }
```

Since *name* is a class data member, we need to manually deallocate the memory allocated for *name* to avoid memory leakage.

Problem in the Getter Function

Object Orientation
Xbject Orientation

```

1  class Student{
2      char* name;
3  public:
4      void setName(char* n);    // Prototype
5      char* getName(){return name;}
6      ~Student(){delete name;}
7  };
8  int main(){
9      Student s;
10     char array[20] = "Object Orientation";
11     s.setName(array);    cout<<s.getName()<<endl;
12     char* p = s.getName();
13     *p = 'X';    // Encapsulation violation
14     cout<<s.getName()<<endl;
15     return 0;
16 }
```

Solution

```
1 class Student{
2     char* name;
3     public:
4     void setName(char* n);
5     char* getName(); // Prototype
6     ~Student(){delete name;}
7 };
8 int main(){
9     Student s;
10    char array[20] = "Object Orientation";
11    s.setName(array); cout<<s.getName()<<endl;
12    char* p = s.getName();
13    *p = 'X'; // Attempt to violate
14    cout<<s.getName()<<endl;
15    return 0;
16 }
```

Object Orientation
Object Orientation

Solution (Cont.)

```
1 char* Student::getName(){           // Definition
2     int i, length = 1;
3     char* temp;
4
5     // Implementation of strlen(n)
6     for(i=0; name[i] != '\0'; ++i) length++;
7     temp = new char[length]; // A temporary location
8
9     // Implementation of strcpy(name, n)
10    for(i=0; i<length; ++i) temp[i] = name[i];
11    temp[length] = '\0';
12
13    return temp; // a copy of the original contents
14 }
```

Since *temp* is a local variable, its memory will be automatically deallocated when the function returns.